FIRED FESTIVAL · AI HARNESS · 24-HOUR BUILD CHALLENGE

Policy-to-Proof

An AI harness that proves a system is safe enough to ship.

Point it at infrastructure-as-code and a vague requirement — *"make this SOC 2 ready before deploy"* — becomes executable checks, timestamped evidence, suggested PR fixes, and a CISO-ready verdict for the engineer who deploys, the compliance lead who must prove the posture, and the auditor who verifies it. **Operating principle:** deterministic scanners perform every check; the agent performs only judgment, so the harness cannot certify a control it cannot prove. The reference anatomy (Loop · Tools · Guardrails · Observability) is the skeleton; the four required modules hang off it.

THE FOUR PILLARS — deck anatomy houses each required module

Deck pillar	Delivers (required)	In Policy-to-Proof
1 · Loop	Checkpoints	Bounded scan loop; four ordered gates (parse → control → evidence → remediation) return PASS / FAIL / N⁄A, persisted & replayable; behaviour changes on gate feedback; caps on turns, tokens, time.
2 · Tools	Material handling	Deterministic scanners (checkov / tfsec / gitleaks) as typed tools — clean corpus in, structured evidence packet out; swappable worker behind evaluate(), any model, no harness change.
3 · Guardrails	Guardrails	Declared in controls.yaml — input (strip injection, size-limit), action (read-only target, allow-list, never auto-apply fixes), output (no PASS without evidence; redact secrets) + turn / token / spend limits.
4 · Observability	Alarms	OTel span per check; named alarms {type · context · severity · action}, e.g. SECRET_EXPOSED → halt; tracks p95 latency, $/run, tool err%, eval pass-rate.

HOW IT RUNS — one Terraform scan, end to end

```
requirement → [guardrail IN: strip / validate / redact secrets]
  → parse_gate → control_gate (scanners) → evidence_gate (provenance) → remediation_gate (diff+lint)
  → [guardrail OUT: no unproven PASS] → [observability: span per step]
  → risk score | control matrix | evidence packet | PR diffs | CISO verdict
any gate can raise an alarm; SECRET_EXPOSED halts the run before a single PASS is issued
```

WORKED EXAMPLE

Input: a main.tf with an unencrypted RDS instance, an IAM policy using Action = "*", and a hardcoded AWS key.
Fires: gitleaks → **SECRET_EXPOSED** (critical) halts; checkov → CONTROL_FAILED ×2 (encryption, least-privilege, high), each pinned to file:line.
Output: risk 34/100; matrix 3 fail / 2 pass; PR diff sets storage_encrypted = true and scopes the IAM action; CISO verdict: **not ship-ready — 1 critical, 2 high.**

CONTROL SET — v1

- Access logging — CloudTrail, S3 logs, VPC flow logs
- Least privilege — no wildcard (*) IAM
- Secrets mgmt — no hardcoded creds; KMS present
- Encryption — at rest (S3/RDS/EBS) & in transit
- CI/CD — the harness wired in as a deploy gate

RUBRIC COVERAGE

MUST — four separable modules; behaviour changes on gate feedback; guardrails declared; checkpoints pass/fail; alarms structured; runs on a real Terraform repo; HARNESS.md.
SHOULD — swappable worker; checkpoint replay; human-in-the-loop on unmapped / missing-evidence / low-confidence.
BONUS — swap a second worker live to prove portability.

STACK & DELIVERABLES

Python harness — guardrails/ checkpoints/ materials/ scanners/ alarms/ worker/. OpenTelemetry tracing. Worker = Anthropic API behind evaluate(). UI: FastAPI / Streamlit; deploy on Render or Fly.
Demo: five-minute video _____

Start with the loop. Harden from there.

The model is a commodity invoked by the harness; the harness is the durable engineering, and the point at which trust is established: the moment before deployment.